

Tarea 03
Grafos Algoritmos

El objetivo es desarrollar una aplicación de consola en Lenguaje C/C++ que actúe como un "GPS básico" para optimizar el transporte entre ciudades. El sistema leerá la configuración de calles desde un archivo de texto, permitirá actualizaciones dinámicas en tiempo real mediante un menú interactivo y calculará las rutas óptimas utilizando el algoritmo de **Dijkstra** (hallar distancias mas cortas) y el algoritmo de **Prim o Kruskal** para el tendido de fibra óptica para comunicación digital mejorar las audiencias en internet entre las ciudades (redes sociales, buscadores, sitios web, blogs, etc). Una correcta comunicación web se enfoca en la velocidad de acceso, la usabilidad y la creación de contenidos adaptados al consumo en línea.

Como base para la tarea se adjunta los programas que les se sera de ayuda y en donde deben completar la tarea, dependiendo del análisis que dedique a ello, *grafosListaAdyaFile.cpp* con algunas operaciones básicas de grafos en lista de adyacencia; *algoDijkstraFile v0.cpp* con algunas implementaciones de algoritmo de Dijkstra. que requiere trabajo de formalización y buenas practicas.

Requerimientos funcionales.

Debe estructurarse mediante menú cíclico con las siguientes opciones.

- 1 Cargar la relación de ciudades. Leer el archivo de texto con formato (texto, texto, entero) e inicializar el grafo mediante Listas de Adyacencia con una correspondencia los nombres a indices numéricos.
 - 1.1 El archivo de texto debe contener al menos de 20 a 30 distritos limeños con una distancia real en kilómetros aproximada. Opcionalmente la data la puede obtener utilizando Google Maps u otras para hacer mas real la aplicación.
- 2 Visualizar la red de distritos limeños. Imprime de forma clara la relación de distritos registrados y el listado de conexiones de cada uno de los distrito con sus respectivos costos o distancias.
- 3 Agregar una cenexion manual. Permitir al usuario ingresar por teclado el distrito origen, distrito destino y el nuevo costo o distancia. La aplicación debe validar si los distritos existen y volver a ingresar si fuese necesario sin alterar el file original.
- 4 Calcular la ruta optima (Dijkstra). Solicitar los distritos origen y destino validando las entradas, calcular la distancia mínima y mostrar el camino/ruta detallado en forma detallada.
- 5 Diseñar la red de conectividad mínima. Calcular el Árbol de Expansión Mínima de la red del sistema empleando Prim o Kruskal. Debe mostrar las carreteras que se deben priorizar para interconectar todas los distritos con el menor costo total que simula el cableado de fibra óptica.
- 6 Salir de la Aplicación. Liberar toda la memoria dinámica utilizada utilizando free y cerrar el programa adecuadamente.

Restricciones. Se debe utilizar Lenguaje C, la aplicación base donde debe implementar utilizando estructuras enlazadas y apuntadores dinámicos con malloc. El sistema no debe colgarse si el usuario ingresa nombre con errores ortográficos o distritos inexistentes. Evite problemas con la RAM. Antes de cargar el grafo y finalizar la aplicación la memoria RAM debe limpiarse nodo a nodo.

Entregables:

Crear una capeta *proPaternoMaternoNombreG03*, una subcarpeta *código* y otra *info*. En *código* adjunte código fuente, archivos de prueba de 5 ciudades y mínimo 20 distritos de Lima (capital) y *info* el informe word/write en no mas de 5 paginas(Intro, objetivos, solución (capturas de pantalla), conclusiones referencias)

Ejemplo: *proGonzalesPerezPepeG03*

Entregar hasta la 2200 horas de 05.julio.2026 tome sus precauciones no habrá extensión de hora ni de fecha de entrega.

Rubrica

No	Criterio	Descripcion	Puntaje
1	Arquitectura de Sw	Estructura de datos, metodología modular, prog principal solo debe solo llamadas a subprogramas.	4
2	Validación y Robustez	Control de errores de files, errores de texto, mecanismo para evitar se cuelgue la aplicación.	4
3	Algo Dijkstra	Cálculo de costo mínimo y despliegue del camino exacto mediante el rastreo recursivo de predecesores.	4
4	Algo Prim	Correcta codificación de Prim o de Kruskal para hallar el árbol de costo mínimo sobre las mismas estructuras de grafo	5
5	Buenas practicas	Liberación de memoria, identificadores descriptivos, comentarios pertinentes. formato de salida utilizando instrucciones adecuadas.	3
	TOTAL		20

Referencias

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley. Joyanes, L. (2016) Fundamentos De Programación. Algoritmos, Estructura de Datos y Objetos. Madrid, España: McGraw-Hill, 4ta ed.

Jayanes L, Zahonero I. (2008) Estructuras de datos en Java. McGraw-Hill. Madrid Espana.

Cairo O. y Guardati S. (2006) Estructura de datos McGraw-Hill/Interamericana de EditresSA. 3Ra Ed. Mexico D.F.